

Valueless Variants Considered Harmful

I. Summary

This paper argues in section III that when `variant`'s contained types have `noexcept` move constructors, `variant` *shall* never be valueless, that is, the specification should statically guarantee that `valueless_by_exception()` will never return `true`.

It then proposes, in section IV, a way to extend these guarantees to types such as `std::list` that are not guaranteed to have a `noexcept` move constructor, by introducing the concept of *pilfering constructor*.

Finally, in section V, it ventures a suggestion that at this point, we might as well get rid of `valueless_by_exception` altogether.

II. Introduction

The `variant` consensus at present, reflected in D0088R2.17, its most recent specification at time of writing, can be summarized by the following quotes by Tony Van Eerd:

The current variant basically does everything:

- "never" empty
- no extra memory
- etc

OK, what doesn't it do.

It doesn't offer the strong exception guarantee if the move constructor throws.

That's it.

Now ask yourself:

- how many move constructors are not `noexcept`
- for move constructors that are not `noexcept`, how many ever actually throw?

That second part is important. Basically it never happens, or if it does, you have bigger problems because you are so out of memory that you can't allocate 32 bytes for a node or something. (ie list constructor in some implementations). We are trying to solve a problem that doesn't really need a **solution**, but does need an **answer**, and one better than UB.

So, yeah, double buffering would be a solution, but you are paying a cost for something that never actually happens. Or you add an empty state, and pay the programmer cost (of dealing with empty) for something that never happens. Or you have two variants, and pay the cost of confusion, for something that never happens.

We, as a committee, want perfection and want to be concerned about the corner cases, but in reality, they never happen.

and David Sankel:

If a developer conforms to the sane subset of C++ where move constructors don't throw, then their variants won't get into the valueless state.

In other words, the current consensus acknowledges that `variant` getting in the valueless state is undesirable, and I absolutely agree.

I however have two objections to the above quotes. First, the statements

- "variant doesn't offer the strong exception guarantee if the move constructor throws."
- "If a developer conforms to the sane subset of C++ where move constructors don't throw, then their variants won't get into the valueless state."

are simply false under the current specification. `variant` habitually does not give the strong exception guarantee on assignment, and can go into the valueless state, even when the move constructors of the contained types don't throw.

Second, I do not consider the "will never happen" philosophy good enough for a standard C++ component. It's fine for a TS, which is meant to be experimental, gather experience, and can be fixed when a defect is discovered without regard to code being broken. Once a component gets into the C++ standard, changing it becomes very hard.

"Will never happen" can be, pragmatically speaking, the right strategy under many circumstances, when the cost of the solution outweighs the cost of the problem. It does have its disadvantages though, one of which is that "never happen" scenarios, being extremely rare, are never tested and therefore tend to occur in production when their costs are high. (Insert Ariane 5 reference here.)

That is why some programmers prefer to rely on static (compile-time) guarantees that the scenarios that "never happen" do indeed never happen, and **it is my opinion that it is a requirement for a C++17 variant to provide such a static, compile-time, guarantee that it will never go into a valueless state if certain restrictions, which can be checked at compile time, are met.**

Axel Naumann prefers a different approach:

I want the wording to allow your suggestions without requiring them.

but I respectfully disagree. "Allowed but not required" is not good enough. First, "allowed but not required" does not give compile-time guarantees. Second, it hampers portability. Third, the wording is subtle and this makes it possible for what is intended to allow but not require to turn out to disallow.

III. Never Valueless

The previous section concluded that `variant` should provide a static, compile-time, guarantee that it will never go into a valueless state if certain restrictions, which can be checked at compile time, are met. What should those restrictions be?

Unsurprisingly, and in agreement with the existing prevailing opinion, that the move constructors of the contained types are `noexcept`.

(Note that move *assignments* being `noexcept` is not required. One might naively think that a type that has a `noexcept` move constructor would also have a `noexcept` move assignment so we might as well require that with no loss of generality, but as usual, the standard library has a surprise for us, in that `std::vector`'s move assignment is not necessarily `noexcept`.)

What do we need to change in D0088R2.17 to fulfill this requirement?

There are only two ways for a `variant` to become valueless: assignment and `emplace`. Let's consider all their variations in turn.

variant& operator=(const variant& rhs);

Effects: **Let `j` be `rhs.index()`.**

- If neither `*this` nor `rhs` holds a value, there is no effect. Otherwise,
- if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value. Otherwise,
- if `index() == rhs.index()+j` **&& is nothrow copy assignable v<T j>**, assigns the value contained in `rhs` to the value contained in `*this`. Otherwise,
- **if `index() == j` **&& is nothrow move assignable v<T j>**, copies the value contained in `rhs` to a temporary `TMP`, then assigns `std::forward<T j>(TMP)` to the value contained in `*this`. Otherwise,**

- if `index() == j` && `!is_nothrow_move_constructible_v<T j>`, assigns the value contained in `rhs` to the value contained in `*this`. Otherwise,
- copies the value contained in `rhs` to a temporary `TMP`, then destroys any value contained in `*this`. Sets `*this` to hold the same alternative index as `rhs` and initializes the value contained in `*this` as if direct-non-list-initializing an object of type `T_j` with `std::forward<T_j>(TMP)`, with `TMP` being the temporary and `j` being `rhs.index()`.

Returns: `*this`.

Postconditions: `index() == rhs.index()`

Remarks: This function shall not participate in overload resolution unless `is_copy_constructible_v<T_i>` && `is_move_constructible_v<T_i>` && `is_copy_assignable_v<T_i>` is true for all `i`.

- If an exception is thrown during the call to `T_j`'s copy assignment, the state of the contained value is as defined by the exception safety guarantee of `T_j`'s copy assignment; `index()` will be `j`.
- If an exception is thrown during the call to `T_j`'s copy constructor (with `j` being `rhs.index()`), `*this` will remain unchanged.
- If an exception is thrown during the call to `T_j`'s move constructor, the variant will hold no value.

These changes make sure that when the move constructor of `T_j` is `noexcept`, the assignment will never put the variant into the valueless state and that it will provide the strong exception safety guarantee.

variant& operator=(variant&& rhs) noexcept(see below);

Effects: `Let j be rhs.index()`.

- If neither `*this` nor `rhs` holds a value, there is no effect. Otherwise
- if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value. Otherwise,
- if `index() == rhs.index()` && `(is_nothrow_move_assignable_v<T j> || !is_nothrow_move_constructible_v<T j>)`, assigns `std::forward<T_j>(get<j>(rhs))` to the value contained in `*this`, with `j` being `index()`. Otherwise,
- destroys any value contained in `*this`. Sets `*this` to hold the same alternative index as `rhs` and initializes the value contained in `*this` as if direct-non-list-initializing an object of type `T_j` with `std::forward<T_j>(get<j>(rhs))` with `j` being `rhs.index()`.

Returns: `*this`.

Remarks: This function shall not participate in overload resolution unless `is_move_constructible_v<T_i>` && `is_move_assignable_v<T_i>` is true for all `i`. The expression inside `noexcept` is equivalent to: `is_nothrow_move_constructible_v<T_i>` && `is_nothrow_move_assignable_v<T_i>` for all `i`.

- If an exception is thrown during the call to `T_j`'s move constructor (with `j` being `rhs.index()`), the variant will hold no value.
- If an exception is thrown during the call to `T_j`'s move assignment, the state of the contained value is as defined by the exception safety guarantee of `T_j`'s move assignment; `index()` will be `j`.

As above: These changes make sure that when the move constructor of `T_j` is `noexcept`, the assignment will never put the variant into the valueless state and that it will provide the strong exception safety guarantee with respect to `*this`.

template <class T> variant& operator=(T&& t) noexcept(see below);

Effects: `operator=(variant(std::forward<T>(t)))`.

Returns: `*this`.

Remarks: This function shall not participate in overload resolution unless `is_same_v<decay_t<T>, variant>` is false and unless `variant(std::forward<T>(t))` is a valid expression. The expression inside `noexcept` is `noexcept(operator=(variant(std::forward<T>(t))))`.

The existing specification of this assignment needlessly duplicates the wording in `variant::variant(T&& t)` that selects the alternative using overload resolution, and does not provide any non-valueless guarantees due to initializing directly from `t` instead of using the potentially `noexcept` move constructor of the selected contained type. I have opted to use the cleanest fix in the above suggested wording. It's possible to expand the expression `operator=(variant(std::forward<T>(t)))` into the specification, but the only thing that this gains is collapsing two adjacent move constructors calls into one, and the implementation is permitted to do this anyway under the as-if rule, so the benefits do not outweigh the costs of using the more complicated wording, with the associated possibility of getting it wrong.

Or, another option is to remove this assignment operator altogether, which would be equivalent to this specification.

template <size_t I, class... Args> void emplace(Args&&... args);

Requires: `I < sizeof...(Types)`

Effects: Destroys the currently contained value if `valueless_by_exception()` is false. Then direct-initializes the contained value as if constructing a value of type `T_I` with the arguments `std::forward<Args>(args)...`.

- If `is_nothrow_constructible_v<T_I, Args&&...> || !is_nothrow_move_constructible_v<T_I>`, destroys the currently contained value if `valueless_by_exception()` is false and direct-initializes the contained value as if constructing a value of type `T_I` with the arguments `std::forward<Args>(args)...`. Otherwise,
- direct-initializes a temporary `TMP` of type `T_I` with the arguments `std::forward<Args>(args)...`, destroys the currently contained value if `valueless_by_exception()` is false and direct-initializes the contained value as if constructing a value of type `T_I` with the argument `std::forward<T_I>(TMP)`.

Postcondition: `index()` is `I`.

Throws: Any exception thrown during the initialization of the contained value.

Remarks: This function shall not participate in overload resolution unless `is_constructible_v<T_I, Args&&...>` is true. If an exception is thrown during the initialization of the contained value, the variant will not hold a value.

As above: These changes make sure that when the move constructor of `T_I` is `noexcept`, `emplace` will never put the variant into the valueless state and that it will provide the strong exception safety guarantee.

template <size_t I, class U, class... Args> void emplace(initializer_list<U> il, Args&&... args);

Requires: `I < sizeof...(Types)`

Effects: Destroys the currently contained value if `valueless_by_exception()` is false. Then direct-initializes the contained value as if constructing a value of type `T_I` with the arguments `il, std::forward<Args>(args)...`.

- If `is_nothrow_constructible_v<T_I, initializer_list<U>&, Args&&...> || !is_nothrow_move_constructible_v<T_I>`, destroys the currently contained value if `valueless_by_exception()` is false and direct-initializes the contained value as if constructing a value of type `T_I` with the arguments `il, std::forward<Args>(args)...`. Otherwise,
- direct-initializes a temporary `TMP` of type `T_I` with the arguments `il, std::forward<Args>(args)...`, destroys the currently contained value if `valueless_by_exception()` is false and direct-initializes the contained value as if constructing a value of type `T_I` with the argument `std::forward<T_I>(TMP)`.

Postcondition: `index()` is `I`.

Throws: Any exception thrown during the initialization of the contained value.

Remarks: This function shall not participate in overload resolution unless `is_constructible_v<T_I, initializer_list<U>&, Args&&...>` is true. If an exception is thrown during the initialization of the contained value, the variant will not hold a value.

As above.

```
template <class T, class... Args> void emplace(Args&&... args);
```

```
template <class T, class U, class... Args> void emplace(initializer_list<U> il, Args&&... args);
```

These two overloads of `emplace` are specified in terms of the index-based ones, so no changes are required.

```
constexpr bool valueless_by_exception() const noexcept;
```

Effects: Returns `false` if and only if the variant holds a value. [Note: A variant will not hold a value if an exception is thrown from the move constructor of the contained type during a type-changing assignment or emplacement. — end note]

Remarks: This function shall be static and always return false when `is_nothrow_move_constructible_v<T_i>` is true for all `i`. [Note: `static_assert(variant<Types...>::valueless_by_exception() == false);` may be used to verify that a `variant<Types...>` may never become valueless. — end note]

IV. Pilfering

The changes in the preceding section do give us the necessary guarantees in most cases, but there's still a problem with, for example, `variant<int, std::list<int>>`. Under some implementations, every `list` instance allocates a sentinel node, and since `list`'s move constructor needs to leave the moved-from object in a valid state, it can't steal its sentinel node for the new instance, forcing an allocation and therefore precluding `noexcept`. This means that `variant<int, std::list<int>>` would be guaranteed valueless on some implementations and not on others, which is a portability concern.

It so happens that the implementation of `variant` usually moves from an internal temporary that is later destroyed and is invisible to the outside code. A move-constructed `list` could, therefore, steal the sentinel node of this temporary, but there is no standard protocol for doing so.

The implementation of `std::variant` could, of course, detect `std::list` and use some internal constructor instead, and one might argue that a quality implementation ought to do so, but this cannot extend to user-defined types, or even to `std::pair<T, std::list<int>>`.

This section proposes a general mechanism to enable such semi-destructive move construction, after which the moved-from object can be safely destroyed, but is not guaranteed to be usable in any other way. This semi-destructive move is called *pilfering* and is accessed by a constructor with the signature `T::T(std::pilfered<T>) noexcept`, where `std::pilfered<T>` wraps a reference to `T`.

```
template<class T> class pilfered
{
private:
    T& t_;

public:
    explicit constexpr pilfered(T&& t) noexcept: t_(t) {}

    constexpr T& get() const noexcept { return t_; }
    constexpr T* operator->() const noexcept { return std::addressof(t_); }
};
```

and there's also a corresponding type trait `std::is_pilfer_constructible<T>` and a helper function `std::pilfer(t)` which is analogous to `std::move(t)`:

```
template<class T> struct is_pilfer_constructible: std::integral_constant<bool, std::is_nothrow_move_constructible<T>::value || (std::is_nothrow_constructible<T, pilfered<T>>::value && !std::is_nothrow_constructible<T, pilfered<T>>::value)>::value> {
};

template<class T> constexpr decltype(auto) pilfer(T&& t) noexcept
{
    using U = std::remove_reference_t<T>;
    return std::conditional_t<std::is_nothrow_move_constructible<U>::value || !is_pilfer_constructible<U>::value, U&, pilfered<U>>(std::move(t));
}
```

`is_pilfer_constructible<T>` reports true when `T` has either a noexcept move constructor or a noexcept pilfering constructor. It checks for construction from `__not_pilfered`, which has the same definition as `pilfered`, in order to detect false positives caused by types that are constructible from an argument of any type.

`pilfer(t)` returns either an rvalue reference to `t` or a `pilfered<T>` instance that refers to `t`, as appropriate.

In the past I have suggested a pilfering mechanism that uses a function instead of a constructor, but a function-based approach does not compose. A pilfering constructor for

```
struct X
{
    T t;
    U u;
};
```

where τ and u are known to be either `noexcept move constructible` or `pilfer constructible`, can be added via

```
struct X
{
    T t;
    U u;
};

X(std::pilfered<X> r) noexcept: t(std::pilfer(r->t)), u(std::pilfer(r->u)) {}
```

which is analogous to adding an ordinary move constructor.

The case in which `τ` and `u` are not known in advance, such as with `std::pair`, becomes more convoluted because the initialization of the members might throw, and we don't want to define the pilfering constructor in this case (it makes no sense to define a pilfering constructor that is not `noexcept`). `is_pilfer_constructible<T>` can be used to disable the pilfering constructor via SFINAE. One possible implementation of `x`'s pilfering constructor for the template case would be

```
template<class T, class U> struct X
{
    T t;
    U u;

    template<class T2 = T, class U2 = U, class E = int[std::is_pilfer_constructible_v<T2> && std::is_pilfer_constructible_v<U2>? 1: -1]>
    X(std::pilfered<X> r) noexcept: t(std::pilfer(r->t)), u(std::pilfer(r->u)) {}
};
```

`is_pilfer_constructible` is satisfied by either a `noexcept` move constructor or a `noexcept` pilfering constructor, and the expression `t(std::pilfer(x->t))` will work with either, so a combinatorial explosion does not occur.

The standard wording for pilfering (relative to N4567) is given below.

— Add to the synopsis of header <utility> in [utility] the following:

```
// 20.x. Pilfering
template<class T> class pilfered;
template<class T> constexpr decltype(auto) pilfer(T&& t) noexcept;
```

— After [intseq], add new sections [pilfered] and [pilfer]:

20.x Class template pilfered [pilfered]

```
template<class T> class pilfered
{
private:
    T& t;

public:
    explicit constexpr pilfered(T&& t) noexcept;
    constexpr T& get() const noexcept;
    constexpr T* operator->() const noexcept;
};
```

pilfered<T> wraps a reference to **T** and is used as an argument to **T**'s *pilfering constructor*. Pilfering constructors have the form **T::T(pilfered<T>) noexcept** and perform a semi-destructive move. After a call to a pilfering constructor, the moved-from object can be safely destroyed, but cannot be used in any other way.

```
explicit constexpr pilfered(T&& t) noexcept;
```

Effects: Initializes **t** to **t**.

```
constexpr T& get() const noexcept;
```

Returns: **t**.

```
constexpr T* operator->() const noexcept;
```

Returns: `addressof(t)`.

20.x Function template pilfer [pilfer]

```
template<class T> constexpr decltype(auto) pilfer(T&& t) noexcept;
```

Returns: conditional `t<is_nothrow_move_constructible_v<U> || !is_pilfer_constructible_v<U>, U&&, pilfered<U>{move(t)}>`, where **U** is `remove_reference_t<T>`.

— In the synopsis of header <type_traits> [meta.type.synop], in the group of type properties, add the following:

```
template <class T> struct is_nothrow_move_constructible;
template <class T> struct is_pilfer_constructible;
```

```
template <class T> constexpr bool is_nothrow_move_constructible_v
= is_nothrow_move_constructible<T>::value;
template <class T> constexpr bool is_pilfer_constructible_v
= is_pilfer_constructible<T>::value;
```

— In section [meta.unary.prop], add the following paragraph before Table 49:

In the following table, `not_pilfered` is a class template with an unspecified name whose definition is the same as that of `pilfered`.

— In section [meta.unary.prop], add to Table 49 the following row:

<pre>template <class T> struct is_pilfer_constructible;</pre>	For a referenceable type T, the same result as <code>is_nothrow_move_constructible_v<T> (is_nothrow_constructible_v<T, pilfered<T>> && !is_nothrow_constructible_v<T, not_pilfered<T>>)</code>, otherwise false. [Note: <code>not_pilfered</code> is used to avoid false positives caused by types that can be constructed from any argument. — end note]	T shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
---	---	---

— Add to struct `pair` in [pairs.pair] the following constructor:

```
pair(const pair&) = default;
pair(pair&&) = default;
pair(pilfered<pair> r) noexcept;
constexpr pair();
```

— Add to [pairs.pair] the following section:

```
pair(pilfered<pair> r) noexcept;
```

Effects: Initializes *first* with `pilfer(r->first)` and *second* with `pilfer(r->second)`.

Remarks: This constructor shall not participate in overload resolution unless `is_pilfer_constructible<T1>::value && is_pilfer_constructible<T2>::value`.

— Add to class `tuple` in [tuple.tuple] the following constructor:

```
tuple(const tuple&) = default;
tuple(tuple&&) = default;
tuple(pilfered<tuple> u) noexcept;
```

— Add to [tuple.cnstr] the following section:

--

```
tuple(pilfered<tuple> u) noexcept;
```

Effects: For all i , initializes the i^{th} element of `*this` with `get<  $i$ >(u.get())` when T_i is a reference type, `pilfer(get<  $i$ >(u.get()))` otherwise.

Remarks: This constructor shall not participate in overload resolution unless `is_pilfer_constructible<  $T_p$ >::value` is true for all i .

— Add to class `deque` in [deque.overview] the following constructor:

```
deque(const deque& x);  
deque(deque&&);  
deque(pilfered<deque> x) noexcept;
```

— Add to [deque.cons] the following section:

```
deque(pilfered<deque> x) noexcept;
```

Effects: Constructs `*this` from the contents of the object referenced by `x.get()`.

Postconditions: `*this` has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `forward_list` in [forwardlist.overview] the following constructor:

```
forward_list(const forward_list& x);  
forward_list(forward_list&& x);  
forward_list(pilfered<forward_list> x) noexcept;
```

— Add to [forwardlist.cons] the following section:

```
forward_list(pilfered<forward_list> x) noexcept;
```

Effects: Constructs `*this` from the contents of the object referenced by `x.get()`.

Postconditions: `*this` has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `list` in [list.overview] the following constructor:

```
list(const list& x);  
list(list&& x);  
list(pilfered<list> x) noexcept;
```

— Add to [list.cons] the following section:

```
list(pilfered<list> x) noexcept;
```

Effects: Constructs `*this` from the contents of the object referenced by `x.get()`.

Postconditions: `*this` has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `map` in [map.overview] the following constructor:

```
map(const map& x);  
map(map&& x);  
map(pilfered<map> x) noexcept;
```

— Add to [map.cons] the following section:

```
map(pilfered<map> x) noexcept;
```

Effects: Constructs `*this` from the contents of the object referenced by `x.get()`.

Postconditions: `*this` has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `multimap` in [multimap.overview] the following constructor:

```
multimap(const multimap& x);  
multimap(multimap&& x);  
multimap(pilfered<multimap> x) noexcept;
```

— Add to [multimap.cons] the following section:

```
multimap(pilfered<multimap> x) noexcept;
```

Effects: Constructs `*this` from the contents of the object referenced by `x.get()`.

Postconditions: `*this` has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `set` in [set.overview] the following constructor:

```
set(const set& x);
set(set&& x);
set(pilfered<set> x) noexcept;
```

— Add to [set.cons] the following section:

```
set(pilfered<set> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by `x.get()`.

Postconditions: **this* has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `multiset` in [multiset.overview] the following constructor:

```
multiset(const multiset& x);
multiset(multiset&& x);
multiset(pilfered<multiset> x) noexcept;
```

— Add to [multiset.cons] the following section:

```
multiset(pilfered<multiset> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by `x.get()`.

Postconditions: **this* has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `unordered_map` in [unord.map.overview] the following constructor:

```
unordered_map(const unordered_map&);
unordered_map(unordered_map&&);
unordered_map(pilfered<unordered_map> x) noexcept;
```

— Add to [unord.map.cnstr] the following section:

```
unordered_map(pilfered<unordered_map> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by `x.get()`.

Postconditions: **this* has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `unordered_multimap` in [unord.multimap.overview] the following constructor:

```
unordered_multimap(const unordered_multimap&);
unordered_multimap(unordered_multimap&&);
unordered_multimap(pilfered<unordered_multimap> x) noexcept;
```

— Add to [unord.multimap.cnstr] the following section:

```
unordered_multimap(pilfered<unordered_multimap> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by `x.get()`.

Postconditions: **this* has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `unordered_set` in [unord.set.overview] the following constructor:

```
unordered_set(const unordered_set&);
unordered_set(unordered_set&&);
unordered_set(pilfered<unordered_set> x) noexcept;
```

— Add to [unord.set.cnstr] the following section:

```
unordered_set(pilfered<unordered_set> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by `x.get()`.

Postconditions: **this* has the value the object referenced by `x.get()` had before the call.

Remarks: After this constructor, the object referenced by `x.get()` can safely be destroyed. The behavior of any other access to the object referenced by `x.get()` is undefined.

— Add to class `unordered_multiset` in [unord.multiset.overview] the following constructor:

```
unordered_multiset(const unordered_multiset&);
unordered_multiset(unordered_multiset&&);
unordered_multiset(pilfered<unordered_multiset> x) noexcept;
```

— Add to [unord.multiset.cnstr] the following section:

```
unordered_multiset(pilfered<unordered_multiset> x) noexcept;
```

Effects: Constructs **this* from the contents of the object referenced by *x.get()*.

Postconditions: **this* has the value the object referenced by *x.get()* had before the call.

Remarks: After this constructor, the object referenced by *x.get()* can safely be destroyed. The behavior of any other access to the object referenced by *x.get()* is undefined.

Given these changes, the `variant` specification needs to be updated to use `pilfer` constructors, as follows. The differences are relative to the wording in the preceding section.

variant& operator=(const variant& rhs);

Effects: Let *j* be *rhs.index()*.

- If neither **this* nor *rhs* holds a value, there is no effect. Otherwise
- if **this* holds a value but *rhs* does not, destroys the value contained in **this* and sets **this* to not hold a value. Otherwise,
- if *index() == j* && *is_nothrow_copy_assignable_v<T_j>*, assigns the value contained in *rhs* to the value contained in **this*. Otherwise,
- if *index() == j* && *is_nothrow_move_assignable_v<T_j>*, copies the value contained in *rhs* to a temporary *TMP*, then assigns *std::forward<T_j>(TMP)* to the value contained in **this*. Otherwise,
- if *index() == j* && ~~*is_nothrow_move_constructible_v<T_j>*~~ *is pilfer constructible v<T_j>*, assigns the value contained in *rhs* to the value contained in **this*. Otherwise,
- copies the value contained in *rhs* to a temporary *TMP*, then destroys any value contained in **this*. Sets **this* to hold the same alternative index as *rhs* and initializes the value contained in **this* as if direct-non-list-initializing an object of type *T_j* with ~~*std::forward<T_j>(TMP)*~~ *std::pilfer(TMP)*.

Returns: **this*.

Postconditions: *index() == rhs.index()*

Remarks: This function shall not participate in overload resolution unless *is_copy_constructible_v<T_i>* && *is_move_constructible_v<T_i>* && *is_copy_assignable_v<T_i>* is true for all *i*.

- If an exception is thrown during the call to *T_j*'s copy assignment, the state of the contained value is as defined by the exception safety guarantee of *T_j*'s copy assignment; *index()* will be *j*.
- If an exception is thrown during the call to *T_j*'s copy constructor (with *j* being *rhs.index()*), **this* will remain unchanged.
- If an exception is thrown during the call to *T_j*'s move constructor, the *variant* will hold no value.

variant& operator=(variant&& rhs) noexcept(see below);

Effects: Let *j* be *rhs.index()*.

- If neither **this* nor *rhs* holds a value, there is no effect. Otherwise
- if **this* holds a value but *rhs* does not, destroys the value contained in **this* and sets **this* to not hold a value. Otherwise,
- if *index() == j* && (*is_nothrow_move_assignable_v<T_j>* || ~~*is_nothrow_move_constructible_v<T_j>*~~ *is pilfer constructible v<T_j>*), assigns *std::forward<T_j>(get<j>(rhs))* to the value contained in **this*. Otherwise,
- ~~*if is_nothrow_move_constructible_v<T_j>* && *is pilfer constructible v<T_j>*, initializes a temporary *TMP* of type *T_j* from *std::forward<T_j>(get<j>(rhs))*. Destroys any value contained in **this*. Sets **this* to hold the same alternative index as *rhs* and initializes the value contained in **this* as if direct-non-list-initializing an object of type *T_j* with *std::pilfer(TMP)*. Otherwise,~~
- destroys any value contained in **this*. Sets **this* to hold the same alternative index as *rhs* and initializes the value contained in **this* as if direct-non-list-initializing an object of type *T_j* with *std::forward<T_j>(get<j>(rhs))*.

Returns: **this*.

Remarks: This function shall not participate in overload resolution unless *is_move_constructible_v<T_i>* && *is_move_assignable_v<T_i>* is true for all *i*. The expression inside *noexcept* is equivalent to: *is_nothrow_move_constructible_v<T_i>* for all *i*.

- If an exception is thrown during the call to *T_j*'s move constructor *in the last bullet*, the *variant* will hold no value.
- If an exception is thrown during the call to *T_j*'s move assignment, the state of the contained value is as defined by the exception safety guarantee of *T_j*'s move assignment; *index()* will be *j*.

The reason of using `pilfer` only when the type can be pilfered but not `noexcept` moved is because we are not allowed to pilfer directly from *rhs*, as there is no guarantee that outside code will not access the value afterwards. So we need to first move into a temporary, and then pilfer that temporary instead. We could do that in either case, but this would result in two moves instead of one when the type has a `noexcept` move constructor.

template <size_t I, class... Args> void emplace(Args&&... args);

Requires: *I < sizeof...(Types)*

Effects:

- If *is_nothrow_constructible_v<T_I, Args&&...>* || ~~*is_nothrow_move_constructible_v<T_I>*~~ *is pilfer constructible v<T_I>*, destroys the currently contained value if *valueless_by_exception()* is false and direct-initializes the contained value as if constructing a value of type *T_I* with the arguments *std::forward<Args>(args)...* Otherwise,
- direct-initializes a temporary *TMP* of type *T_I* with the arguments *std::forward<Args>(args)...*, destroys the currently contained value if *valueless_by_exception()* is false and direct-initializes the contained value as if constructing a value of type *T_I* with the argument ~~*std::forward<T_I>(TMP)*~~ *std::pilfer(TMP)*.

Postcondition: *index()* is *I*.

Throws: Any exception thrown during the initialization of the contained value.

Remarks: This function shall not participate in overload resolution unless *is_constructible_v<T_I, Args&&...>* is true. If an exception is thrown during the initialization of the contained value, the *variant* will not hold a value.

template <size_t I, class U, class... Args> void emplace(initializer_list<U> il, Args&&... args);

Requires: *I < sizeof...(Types)*

Effects:

- If *is_nothrow_constructible_v<T_I, initializer_list<U>&, Args&&...>* || ~~*is_nothrow_move_constructible_v<T_I>*~~ *is pilfer constructible v<T_I>*, destroys the currently contained value if *valueless_by_exception()* is false and direct-initializes the contained value as if constructing a value of type *T_I* with the arguments *il, std::forward<Args>(args)...* Otherwise,
- direct-initializes a temporary *TMP* of type *T_I* with the arguments *il, std::forward<Args>(args)...*, destroys the currently contained value if *valueless_by_exception()* is false and direct-initializes the contained value as if constructing a value of type *T_I* with the argument ~~*std::forward<T_I>(TMP)*~~ *std::pilfer(TMP)*.

Postcondition: *index()* is *I*.

Throws: Any exception thrown during the initialization of the contained value.

Remarks: This function shall not participate in overload resolution unless *is_constructible_v<T_I, initializer_list<U>&, Args&&...>* is true. If an exception is thrown during the initialization of the contained value, the *variant* will not hold a value.

constexpr bool valueless_by_exception() const noexcept;

Effects: Returns false if and only if the variant holds a value. [Note: A variant will not hold a value if an exception is thrown from the move constructor of the contained type during a type-changing assignment or emplacement. — *end note*]

Remarks: This function shall be static and always return false when `is_nothrow_move_constructible_v<T_i>` is `is_pilfer_constructible_v<T_i>` is true for all *i*. [Note: `static_assert(variant<Types...>::valueless_by_exception() == false);` may be used to verify that a `variant<Types...>` may never become valueless. — *end note*]

void swap(variant& rhs) noexcept(*see below*);

Effects: Let *i* be `index()`. Let *j* be `rhs.index()`.

- if `valueless_by_exception()` && `rhs.valueless_by_exception()` no effect. Otherwise,
- if `index() == rhs.index()`, calls `swap(get<i>(*this), get<i>(rhs))` with *i* being `index()`. Otherwise,
- if `valueless_by_exception()`, constructs a value of type `T_j` into `*this` from `std::pilfer(get<j>(rhs))`, sets `index()` to *j* and destroys the contained value of `rhs`, making it hold no value. Otherwise,
- if `rhs.valueless_by_exception()`, constructs a value of type `T_i` into `rhs` from `std::pilfer(get<i>(*this))`, sets `rhs.index()` to *i* and destroys the contained value of `*this`, making it hold no value. Otherwise,
- if `is_pilfer_constructible_v<T_i>` && `is_pilfer_constructible_v<T_j>`, constructs a temporary `TMP` of type `T_i` from `std::pilfer(get<i>(*this))`, destroys the contained value of `*this`, constructs a value of type `T_j` into `*this` from `std::pilfer(get<j>(rhs))`, sets `index()` to *j*, destroys the contained value of `rhs`, constructs a value of type `T_i` into `rhs` from `std::pilfer(TMP)`, and sets `rhs.index()` to *i*. Otherwise,
- exchanges values of `rhs` and `*this`.

Throws: Any exception thrown by `swap(get<i>(*this), get<i>(rhs))` with *i* being `index()` or variant's move constructor and move assignment operator.

Remarks: This function shall not participate in overload resolution unless all alternative types satisfy the Swappable requirements (17.6.3.2). If an exception is thrown during the call to function `swap(get<i>(*this), get<i>(rhs))`, the states of the contained values of `*this` and of `rhs` are determined by the exception safety guarantee of `swap` for lvalues of `T_i` with *i* being `index()`. If an exception is thrown during the exchange of the values of `*this` and `rhs`, the states of the values of `*this` and of `rhs` are determined by the exception safety guarantee of variant's move constructor and move assignment operator. The expression inside `noexcept` is true when `noexcept(swap(declval<T_k>(), declval<T_k>())) && is_pilfer_constructible_v<T_k>` is true for all *k*, false otherwise.

V. Never Valueless, Really

At this point, we have a variant that is never valueless for types that are either noexcept move constructible or pilfer constructible, which covers a large majority of the use cases. The natural next step is to dispense with `valueless_by_exception` altogether by requiring the contained types to be such. Can we afford to do so?

What use cases demand types that are neither noexcept move- nor pilfer-constructible?

First, there are the legacy C++03 types that have a copy constructor but do not have a move constructor, and for some reason, can't be changed. They can be moved, but this involves a copy, and is not noexcept.

Second, there are the types that are neither copyable nor movable, such as `std::mutex`. An example of a variant instantiated on such types would be `variant<std::mutex, std::recursive_mutex>`.

There is an easy workaround in both cases: instead of putting a prohibited type `T` into the variant, use `unique_ptr<T>` instead. `unique_ptr` is noexcept moveable, and the cost in our case of switching to it is a heap allocation and the need to check for `nullptr`.

The benefit of relegating these two uncommon use cases to the dusty "use `unique_ptr`" folder is that we'll finally be able to get rid of `valueless_by_exception` altogether, thereby simplifying the specification a bit and eliminating the need for the programmers to `static_assert` that their variants can't be valueless.

For a C++17 component, this is also the conservative approach. If these two use cases turn out to be important in practice, we can later either bring back `valueless_by_exception` or provide some other way to address them, without breaking any code. If we instead provide the current, `valueless_by_exception`-possessing variant, we cannot at a later date take that away.

More specifically, if we focus on `variant<std::mutex, std::recursive_mutex>`, we see a use case that is indeed legitimate in that it's using variant as if it were a slightly more convenient union { `std::mutex m; std::recursive_mutex rm;` }. However, this variant does not have much in common with `variant<int, float, std::string>`, in that the latter is Regular and the former decidedly isn't. So if we were in a situation where we already had a standard Regular, never valueless, variant and were faced with the need to support this "convenient union" use case, we would probably take a serious look at the possibility of providing a separate component that addresses this need, instead of making variant potentially valueless.

The case for this change is less strong. I consider the previous two sections a strict and necessary improvement to the existing proposal (in spirit and intent, not specific wording, which may well have defects.) On the "strongly in favor" as +2 ... "strongly against" as -2 spectrum, my current position on section III would be +3, and section IV would be +2.5 unless the standard library is instead changed to require noexcept move constructors on all of its types. This section would only score about +1.7.

Therefore, I will not provide suggested wording for the elimination of `valueless_by_exception` in this revision, although I will in a subsequent revision if discussion and straw polls provide an indication that the working groups are willing to consider such a change.

— end